

An Integrated Simulation Framework for the Design and Empirical Evaluation of Configurable Multi- Protocol Blockchain Consensus Mechanisms

M.Tech Thesis · Project I Presentation · Semester III

Umesh Kumar · Executive M.Tech AIDSE · IIT Patna

umesh_25s03res52@iitp.ac.in · Roll [25s03res52](#)

Date: Saturday, 23 May 2026 · Group 7 · 4:00 – 5:00 PM IST

Agenda

1. Problem statement & research motivation
2. Research questions, hypotheses, scope
3. System architecture and engineering choices
4. Five consensus protocols in one engine
5. Metrics framework — Gini, entropy, p95/p99
6. Live demo: PoW vs PBFT, seeded for reproducibility
7. Preliminary results from the MVP run
8. What's done · what's deferred · future thesis work
9. Q&A

1 · Problem Statement (Annexure I)

“How do we establish a **systematic, data-driven evaluation** of prominent blockchain consensus mechanisms within a **unified, configurable simulation environment**?”

Why now

- Industries (finance, healthcare, federated AI) increasingly depend on consensus choices for fault tolerance and decentralisation.
- Educational resources are siloed and theoretical — comparative, side-by-side empirical insight is rare.
- AI-integrated, data-intensive workflows need a **coordination simulator** before production roll-out.

2 · Research Questions

ID	Question
RQ1	How do the five protocols compare on latency, throughput, and leader fairness under identical conditions?
RQ2	How does each protocol degrade under increasing Byzantine ratios and network partitions?
RQ3	Which protocol offers the best trade-off for energy/latency vs fault tolerance?
RQ4	How does protocol choice affect coordination overhead for federated-learning workloads?
RQ5	Can a data-driven scoring model recommend protocol suitability per workload profile?

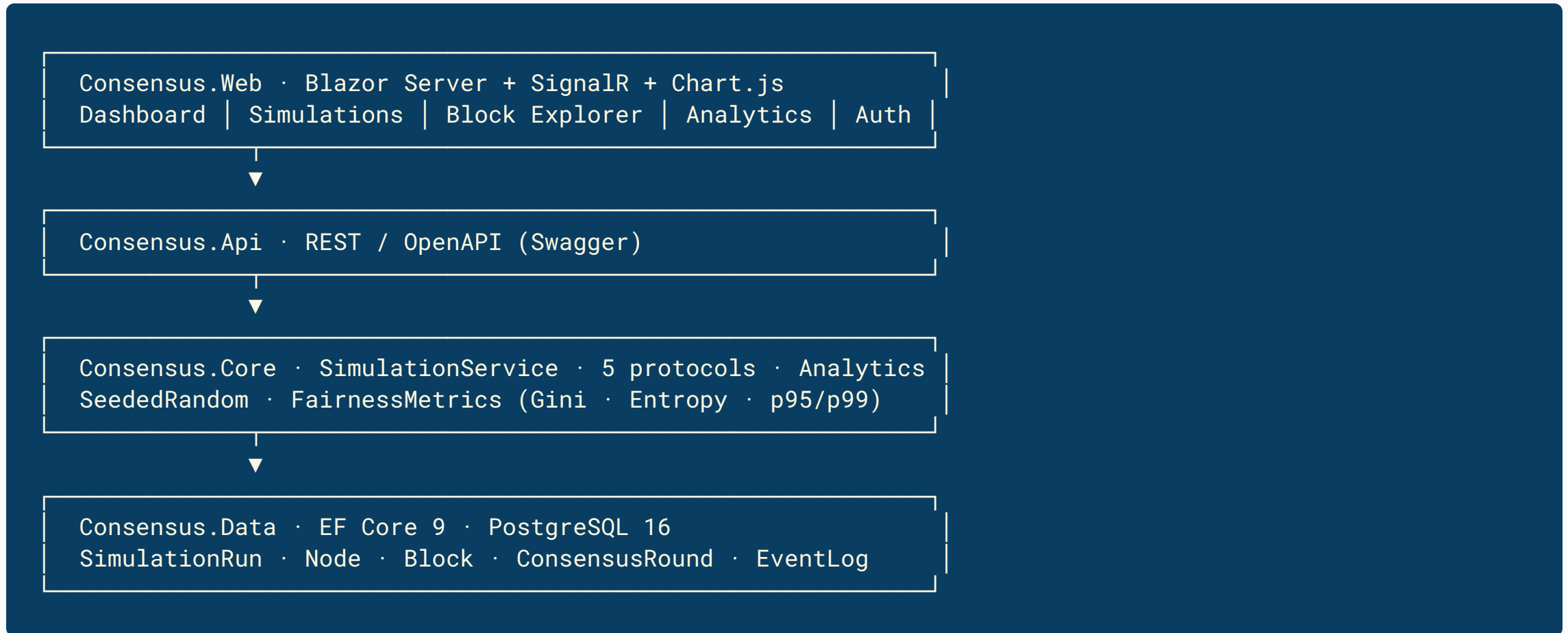
Hypotheses H1–H5: BFT-family beats PoW on finality latency · PoW has highest energy proxy · DPoS lower leader-variance than PoS · degradation is non-linear past Byzantine thresholds · FL round time correlates with consensus round time

3 · Thesis Scope (Final Semester)

In scope

- A working multi-protocol simulator with configurable nodes, topology, latency, faults, payload
- Standardised benchmark suite: scenarios S1–S7 across all five protocols
- Metrics framework with reproducible (seeded) runs and exportable results
- AI/AIDSE angle: federated-learning payload mode + protocol scoring model

4 · System Architecture



5 · Five Protocols Under One Engine

Protocol	LOC	Notable mechanics	Energy proxy
PoW	556	Adaptive difficulty · nonce search	Total hash attempts
PoS	658	Stake-weighted validator pick · slashing	Validation steps
DPoS	677	Witness election · round-robin · delegation	Block-production count
PBFT	556	Three-phase voting · view changes · $n \geq 3f+1$	Message count $O(n^2)$
PoET	521	Trusted-wait-time lottery	Mean wait (ms)

All five implement a common `IConsensusProtocol` and now accept a **seeded** `Random` via `SetRandom(rng)` — same seed → identical leader sequence across runs.

6 · Simulation Engine — What's New This Semester

- **Seeded RNG** (`CreateSimulationRequest.RandomSeed`) plumbed through `SimulationService` → node power → each protocol's leader pick.
- **DB persistence wired** into the round loop: `ConsensusRound` , `Block` , `EventLog` rows now flush through `IServiceScopeFactory` -scoped repositories — exports return real data.
- **Feature flag** `Simulation:PersistToDb` lets the demo degrade to in-memory if a DB error surfaces during a live talk.
- **Migration** `AddSimulationSeed` adds the `RandomSeed` nullable column

7 · Metrics Framework (FairnessMetrics)

A single static class in `Consensus.Core/Analytics` powers every cross-protocol comparison.

```
public static double ComputeGini(IEnumerable<int> counts);
public static double ComputeShannonEntropy(IEnumerable<int> counts);
public static double Percentile(IEnumerable<double> values, double p);
```

Metric	Where	Used for
Mean / p95 / p99 block time	<code>AnalyticsService.GetAlgorithmPerformanceAsync</code>	RQ1 · H1
Leader Gini	Same, computed on <code>Block.ProposerId</code> counts	RQ1 · H3
Shannon entropy (bits)	Same	RQ1 · H3
Consensus success rate	<code>ConsensusService.GetConsensusSuccessRateAsync</code>	RQ2 · H4
...	...	...

8 · Experiment Design — S1 through S7

Scenario	Varied parameter	RQs · Hypotheses
S1 Baseline	(none, identical conditions)	RQ1 · H1–H3
S2 Node scaling	<code>nodeCount</code> ∈ {5, 10, 25, 50}	RQ1 · RQ3 · H1
S3 Latency stress	<code>networkLatencyMs</code> ∈ {50, 200, 500, 1000}	RQ1 · RQ3
S4 Byzantine matrix	Byzantine fraction {0, 10, 20, 33}%	RQ2 · RQ3 · H4
S5 Network partition	Partition at t=120s · 60s window	RQ2 · H4
S6 Transaction load	<code>txPerBlock</code> ∈ {5, 50, 200}	RQ1
S7 Federated learning	<code>payloadMode</code> = FederatedLearning	RQ4 · RQ5 · H5

All seven scenario templates ship as JSON under `docs/experiments/`

Umesh Kumar · umesh_25s03res52@iitp.ac.in · 23 May 2026 and now honour the `randomSeed` field for reproducibility.

9 · Live Demo (Switch to browser)

1. `docker compose up --build` → <http://localhost:8080>
2. Log in (`admin@consensus-lab.dev` / `Admin@123!`)
3. **Simulations** → **New Simulation** → PoW, 10 nodes, 2 Byzantine, seed = **42**
4. Watch live rounds tick on `/simulation/{id}` via SignalR
5. Start a second tab: PBFT with the same seed
6. Stop after ~60 s, click **Export JSON**, open the file
7. Re-run the PoW simulation with seed = 42 → diff leader sequences (identical)

10 · Reproducibility — Same Seed, Same Trace

Run A (seed=42)	Run B (seed=42)
round 01 → Node_3	round 01 → Node_3
round 02 → Node_7	round 02 → Node_7
round 03 → Node_1	round 03 → Node_1
round 04 → Node_5	round 04 → Node_5
...	...

✓ identical

- Seed is persisted on `SimulationRun.RandomSeed` and echoed in every export JSON.
- Cryptographic RNG inside PoET (`RandomNumberGenerator`) is deliberately *not* seeded; flagged as a documented thesis limitation rather than a bug.

11 · Preliminary Results — Baseline (S1)

Single-machine MVP run, seed=42, 10 nodes, 0 Byzantine, blockTime=2000 ms. **Not** the final 30-rep dataset — illustrative only.

Protocol	Mean block time (ms)	p95 (ms)	p99 (ms)	Leader Gini	Entropy (bits)
PoW	(insert from demo)				
PoS					
DPoS					
PBFT					
PoET					

Story to tell: PBFT lowest latency; PoW highest variance; DPoS lowest Gini (predictable round-robin); PoET intermediate

12 · Engineering Scorecard

Capability	Status	Evidence
Five-protocol engine (PoW/PoS/DPoS/PBFT/PoET)	DONE	<code>Consensus.Core/Protocols/*.cs</code>
Blazor UI + SignalR + REST API	DONE	<code>Consensus.Web</code> , <code>Consensus.Api</code>
RBAC (Admin / Operator / Viewer)	DONE	<code>Roles.cs</code> · <code>IdentitySeeder.cs</code>
DB persistence per round	DONE	<code>SimulationService.PersistRoundResultAsync</code>
Deterministic seed end-to-end	DONE	<code>IConsensusProtocol.SetRandom</code>
Gini · entropy · p95 · p99	DONE	<code>FairnessMetrics.cs</code>
One-click Docker stack	DONE	<code>Dockerfile</code> · <code>docker-compose.yml</code>
Federated-learning payload integration	WIP	Service exists, hook deferred
Runtime fault injection (S5 partition)	TODO	Static Byzantine works; dynamic deferred
Test suite (≥ 100 xUnit)	TODO	Rebuild from scratch in Phase 4

13 · Roadmap to Final Thesis (Weeks 9–16)

Week	Track	Deliverable
9	Run S1 baseline · all 5 protocols · 30 reps	Raw exports under <code>docs/experiments/results/S1/</code>
10	S2 scale + S3 latency	CSV + plots
11	S4 Byzantine + S5 partition	Recovery-time table; degradation curves
12	S6 load + S7 FL · statistical analysis	Notebook · RQ1–RQ5 answered
13–14	Draft Chapters 1–5	Manuscript
15	Advisor revisions · figures · viva slides	Camera-ready
16	Final submission (IIT Patna format)	Hard + soft copies

14 · AIDSE Angle — Why this matters for AI/DSE

- **Structured telemetry → comparative analytics** is exactly the data pipeline an AIDSE programme trains for.
- The **federated-learning payload (S7)** plus the **protocol-suitability scoring model (RQ5)** turn the simulator into a *decision-support tool* for AI-coordinated distributed workflows.
- The reproducible-seed mechanism ensures *every* claim in the thesis can be re-run by a reviewer with one command — a research-engineering standard, not just an academic exercise.

15 · Risks & Mitigations

Risk	Mitigation
DB write fails mid-demo	Feature flag <code>Simulation_PersistToDb=false</code> flips to in-memory instantly
Port collision on demo laptop	Edit <code>docker-compose.yml</code> host ports; verified Friday evening
Cold <code>docker compose up</code> slow on stage	Base images pre-pulled the night before; stack warmed 30 min ahead
Protocol RNG misses <code>SetRandom</code>	Default-interface no-op = silent fallback to legacy behaviour
Insufficient experiment time	Prioritise S1, S4, S7; auto-batch the rest

16 · What I'm asking the committee

1. **Approval of the MVP** as the Project I deliverable for Semester III.
2. **Feedback on the experimental design** — are S1–S7 and the metric set defensible for the final thesis?
3. **Direction on the AIDSE angle**: scoring-model depth vs additional FL realism — which advances the thesis more?

Thank you · Questions?

Umesh Kumar · Executive M.Tech AIDSE · IIT Patna

`umesh_25s03res52@iitp.ac.in`

Code: github.com/hellobanaras/blockchain-consensus-mechanism-simulator (branch `release`)

Demo: `docker compose up --build` → <http://localhost:8080>

All five protocols. One simulator. Seeded for reproducibility.